Multigrid Solver with MPI Parallelization

Parallel Solution for the 3D Pressure Poisson Equation

Dennys Huber

February 2, 2026

Practical Parallel Algorithms with MPI (PAMPI)

The Problem with Red-Black SOR

Challenge: Slow Convergence on Fine Grids
Red-Black SOR suffers from frequency-dependent convergence

High-frequency errors

- Oscillate between neighboring points
- **Rapidly smoothed** by RB-SOR
- Few iterations needed

Low-frequency errors

- Smooth variations across grid
- **Very slow** to eliminate
- Dominate on fine grids

Consequence
On large 3D grids RB-SOR requires *many iterations* to converge.

Error Frequency Behavior

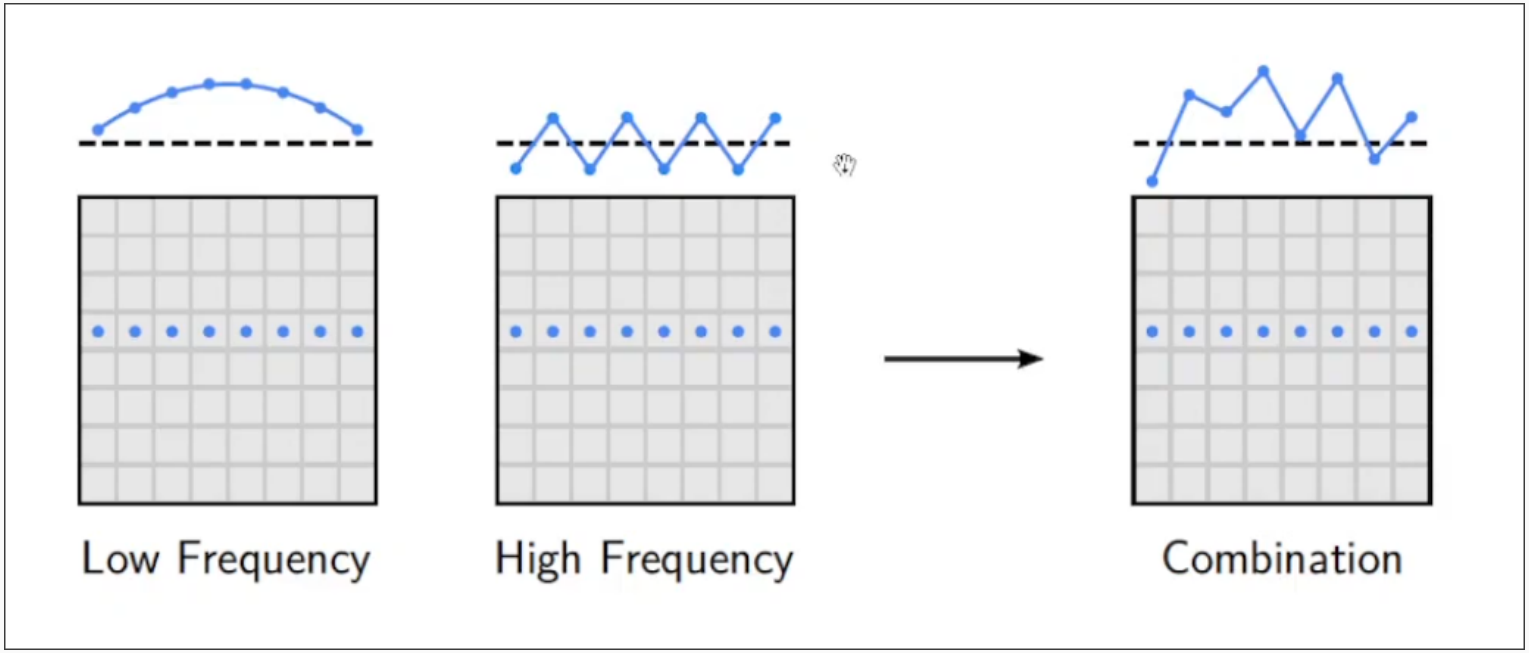


Figure 1: Error frequencies composition

Multigrid Method Overview

Problem
Solving the Pressure Poisson Equation in 3D Navier-Stokes simulations

Solution Approach
Multigrid method with V-cycle:

- **Multiple grid levels** – Coarsens domain hierarchically
- **Pre-smoothing** – Red-Black SOR on fine grid
- **Restriction** – Transfer residual to coarser grid
- **Recursive solve** – Apply V-cycle on coarser level
- **Prolongation** – Interpolate correction to finer grid
- **Post-smoothing** – Refine solution on fine grid

Smoothing: Red-Black SOR

Red-Black Successive Over-Relaxation
Key smoothing operation at each multigrid level

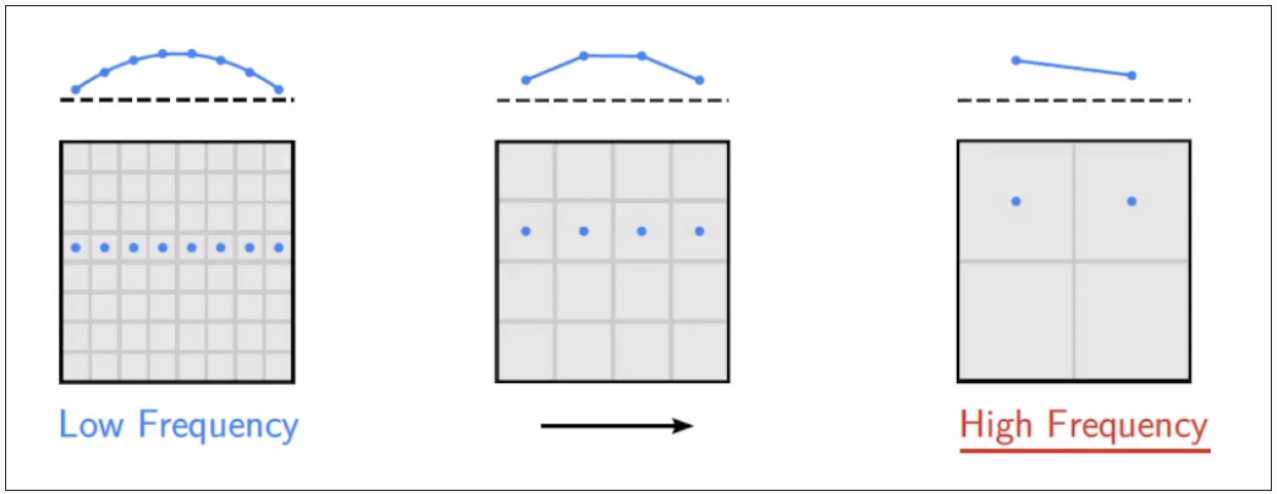
- **Two-color scheme:** Red and black point classification
- **Two passes:** Update red points, then black points
- **7-point stencil:** Finite difference approximation in 3D
- **Relaxation parameter:** Omega factor (typically 1.7–1.9)

Purpose in Multigrid
Rapidly eliminates high-frequency errors while preserving solution quality

Restriction: Fine to Coarse

Restriction (3D)
Transfers residual from fine grid to coarser grid

- 27-point stencil
- Full weighting operator
- Weighted average: $\frac{1}{64}$ factor

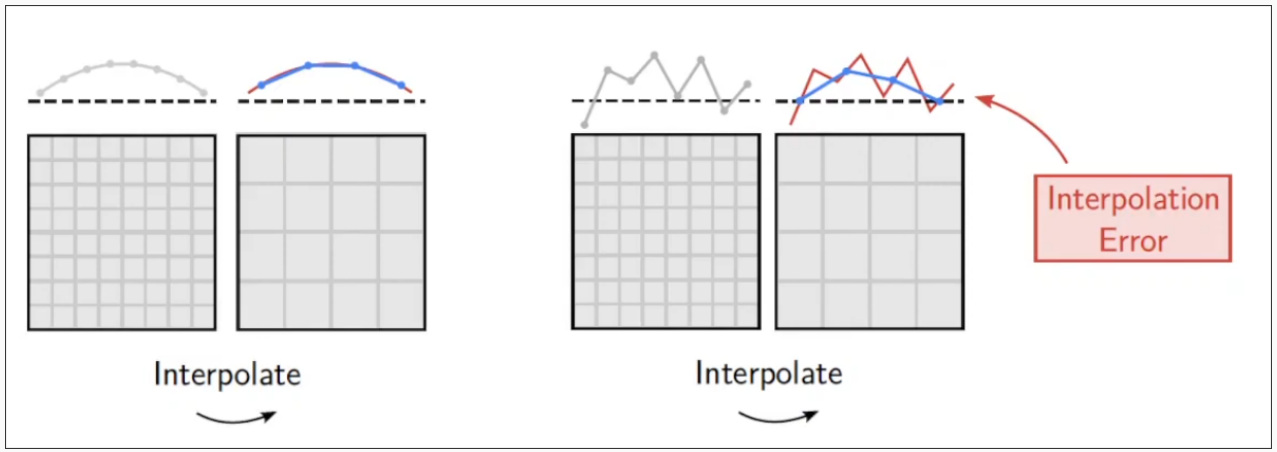


Key Insight
Low-frequency errors on fine grid become high-frequency errors on coarse grid

Prolongation: Coarse to Fine

Prolongation (3D)
Interpolates correction from coarse grid to fine grid

- Linear interpolation
- Coarse to fine transfer
- Bilinear/trilinear scheme



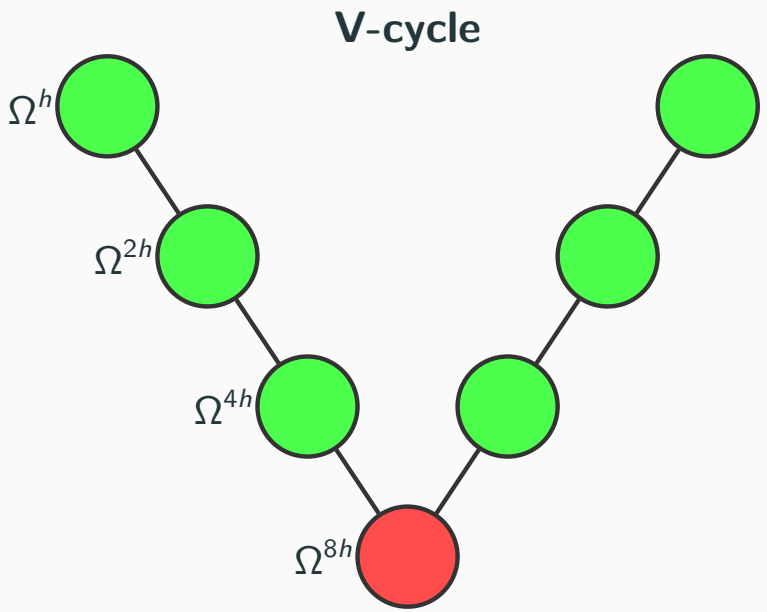
Note
Interpolation may introduce small high-frequency errors, which are eliminated by post-smoothing

Recursive V-Cycle Algorithm

Core Recursive Function

`multiGrid(s, p, rhs, level, imax, jmax, kmax)`

1. **Base case:** Coarsest level $\rightarrow$ smooth and return
2. **Pre-smooth:** Apply relaxation iterations
3. **Calculate residual:** $r = rhs - Ap$
4. **Restrict:** $r_{coarse} = R \cdot r_{fine}$
5. **Recurse:** Solve on coarser grid for error
6. **Prolongate:** $e_{fine} = P \cdot e_{coarse}$
7. **Correct:** $p = p + e$
8. **Post-smooth:** Refine solution



Implementation Details

Data Structure
Arrays for each level: `r[level]` (residual), `e[level]` (error)

Parameters

levels	Number of grid levels (e.g., 3–5)
presmooth	Pre-smoothing iterations
postsmooth	Post-smoothing iterations
omega	SOR relaxation factor (typically 1.7–1.9)

MPI Integration Points

Critical Synchronization
MPI communication is required at several points in the multigrid cycle:

1. **Smoothing:** Exchange ghost cells after each Red-Black pass (`commExchange`)
2. **Restriction:** Exchange residual ghost cells before coarsening
3. **Residual calculation:** Global reduction to sum $\sum r^2$ across all processes
4. **Boundary conditions:** Only processes at physical boundaries set BC

Ghost Cell Exchange: commExchangeLevel

Purpose
Exchange boundary data between neighboring processes at a specific multigrid level

- **MPI Derived Datatypes:** Pre-computed subarrays for each direction and level
- **Send buffer types:** `sbufferTypes[level][direction]` – Extract bulk boundary data
- **Receive buffer types:** `rbufferTypes[level][direction]` – Target ghost cells
- **All-to-all exchange:** `MPI_Neighbor_alltoallw` – Simultaneous communication with all 6 neighbors

Level-Dependent Communication

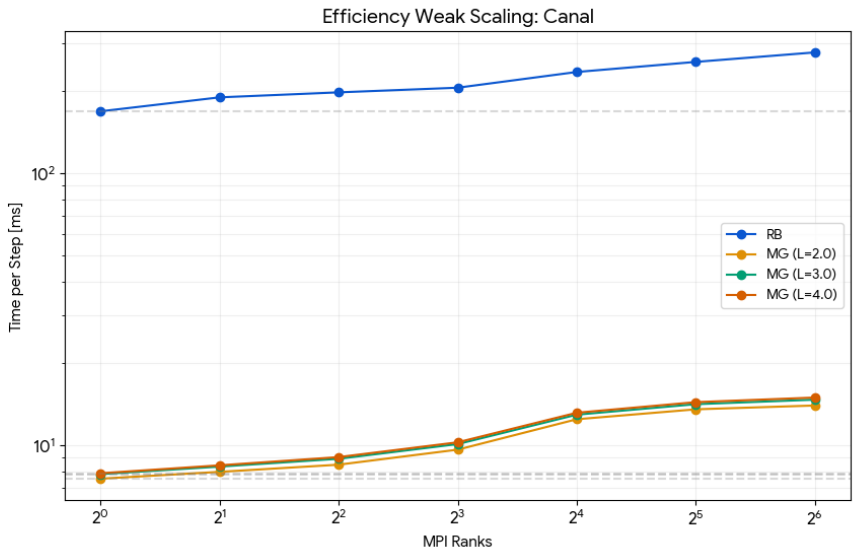
Multi-level MPI Datatypes
Each multigrid level has its own set of communication types

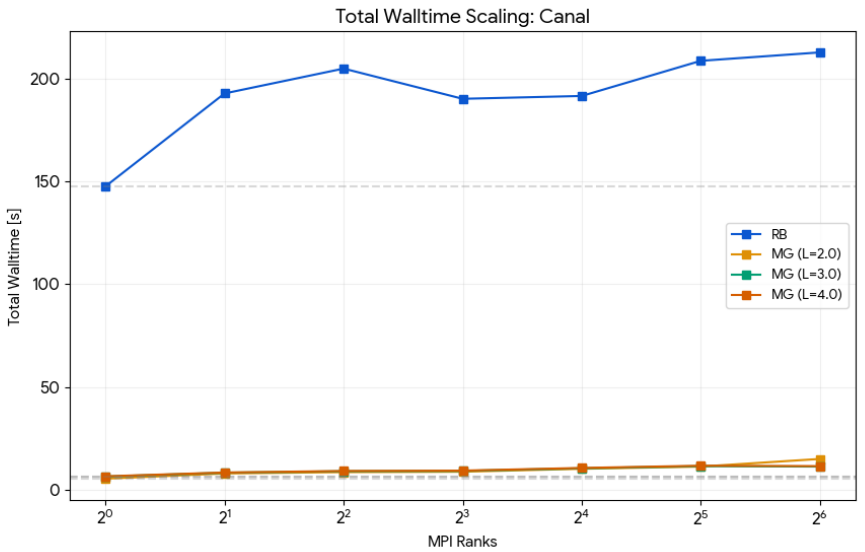
- **Grid hierarchy:** Coarser levels have smaller local domains
- **Type creation:** Setup during `commSetupMG(c, levels)`
- **Dimension halving:** Each level: `imaxLvl /= 2`, `jmaxLvl /= 2`, `kmaxLvl /= 2`

Communication at Different Levels

- Level 0 (finest): Full-size ghost exchange
- Level 1: Half-size ghost exchange
- Level 2 (coarsest): Quarter-size ghost exchange

Experiment: Canal Flow





Experiment: Driven Cavity